Pfun 0.1 Language Manual

Pfun is a scripting language that blends two evaluation models in a single file. Pure functions are lazily evaluated, memoized, and forbidden from producing side effects. Procedures are strictly evaluated, can read and write, and can call functions — but functions can never call procedures. This boundary is enforced at runtime: cross it and you get an error immediately. Everything else — lists, records, pattern matching, infinite lazy sequences, modules — sits on top of this foundation.

Contents

1. Running Pfun bashnpm start script.pf # run a .pf file npm run repl # start the pure functional REPL The REPL runs in pure mode — var, proc, and all I/O are disabled. Lines accumulate until you end one with ?, which triggers evaluation. Type exit or quit to leave.

2. Comments // single-line comment

/ *multi-line comment* /

1. Values and Types Pfun has the following primitive value types: TypeExampleNotesInteger42, 0, -7Arbitrary-precision (BigInt). No floats.Booleantrue, falseString"hello"Immutable sequence of charsChar'a', '\n'Distinct from string — not just a 1-char string Char escape sequences: '\n' (newline), '\t' (tab), '\' (backslash), '\'' (single quote). String escape sequences: \", \, \n, \t. Inside printf strings, { and } produce literal braces. Chars are not strings. 'a' == "a" evaluates to false. let c = 'A'; let s = "A"; println(c == s); // false println(c == 'A'); // true

2. Variables: let and var Pfun has two binding forms with completely different semantics. let — immutable, lazy let x = 10; let greeting = "hello"; let result = some_expensive_function(x);

The right-hand side is not evaluated until the value is first used. Once bound, the name cannot be reassigned. Use let everywhere in pure functional code.

let x = 10; x = 20; // [Name] Error on line 2/ch1: // Cannot assign to immutable variable 'x'. // x = 20; // ^ // // x = 10 var — mutable, strict var counter = 0; counter = counter + 1;

The right-hand side is evaluated immediately. Can be reassigned at any time. Only allowed in procedures and at the top level — attempting to use var inside a pure function is a purity error. Dictionaries must use var (they are inherently mutable).

function bad(x) { var y = x + 1; return y; } // [Purity] Error on line 2/ch5: // Functions cannot use 'var': side-effectful mutation is not // allowed in pure functions. // var y = x + 1; // ^

1. Operators Arithmetic 10 + 3 // 13 10 - 3 // 7 10 * 3 // 30 10 / 3 // 3 (integer division) 10 % 3 // 1 (modulo) All arithmetic operates on integers. There are no floating-point numbers in Pfun 0.1. Dividing by zero produces a [DivideByZero] error. Comparison a == b // equality a != b // inequality a < b a > b a <= b a >= b Comparison operators work on integers and booleans. String and char equality use value equality ("abc" == "abc" is true). Logical a && b // boolean and (short-circuits) a || b // boolean or (short-circuits) !a // boolean not String concatenation The + operator concatenates when either side is a string or char: "Hello" + ", " + "world" // "Hello, world" 'H' + "ello" // "Hello" "count: " + 42 // "count: 42" Ternary condition ? value_if_true : value_if_false let max = a > b ? a : b; let label = n == 1 ? "item" : "items"; The ternary is an expression, not a statement — it produces a value and can appear anywhere a value is expected. Only the selected branch is evaluated (lazy).

2. Control Flow if / then / else if condition then statement if condition then statement else statement if a < b then { println("a is less"); } else { println("b is less or equal"); } The condition must be a boolean. Blocks { } group multiple statements. The else branch is optional. if is a statement, not an expression — use a ternary ? : when you need an expression. Single-statement form (no braces needed): if n == 0 then return "zero" else return "nonzero"; Blocks Braces create a block of statements. The last expression in a block is its value when used as a match arm body: let result = match x { | Some s -> { let doubled = s.value * 2; doubled + 1 } | None -> 0 };

3. Functions function name(param1, param2) { return expression; } Functions are pure: they cannot use print, var, call procedures, or mutate dicts. In exchange, they are lazily evaluated (arguments are not forced until used) and memoized (the result for a given set of arguments is cached and reused on subsequent identical calls). function add(x, y) { return x + y; }

println(add(3, 4)); // 7 A function body can contain multiple statements, but only let, return, type declarations, and pure expression statements are allowed. The last expression before return is not automatically returned — you must use return explicitly. function clamp(value, lo, hi) { return value < lo ? lo : (value > hi ? hi : value); }

println(clamp(15, 0, 10)); // 10 println(clamp(5, 0, 10)); // 5 Recursion Functions can call themselves freely. See Section 10 for tail-call optimisation. function factorial(n, acc) { if n <= 1 then return acc else factorial(n - 1, n * acc); }

println(factorial(10, 1)); // 3628800

1. Procedures proc name(param1, param2) { statements; } Procedures are impure: they can use print, var, read input, write files, and mutate dicts. Arguments are strictly evaluated (forced immediately on call). Procedures are not memoized. proc greet(name) { println("Hello, " + name + "!"); }

greet("Alice"); // Hello, Alice! Procedures can call pure functions: function double(n) { return n * 2; }

proc printDouble(n) { println(double(n)); }

printDouble(7); // 14 The reverse is forbidden — functions cannot call procedures: proc sideEffect() { println("oops"); }

function bad(x) { return sideEffect(); } // [Purity] Error: // Functions cannot call procedures: 'sideEffect' is a procedure.

1. Lambdas fn param => expression fn param1, param2 => expression Lambdas are anonymous pure functions. They are always pure by construction — there is no proc equivalent. A lambda body is a single expression (no multi-statement body). let double = fn x => x * 2; let add = fn x, y => x + y;

println(double(5)); // 10 println(add(3, 4)); // 7 Lambdas are most useful as arguments to higher-order functions: let evens = filter(fn x => x % 2 == 0, [1, 2, 3, 4, 5]); let scaled = map(fn x => x * 10, evens); println(scaled); // [20, 40] You can also bind a lambda with let: let multiply = fn x, y => x * y; println(multiply(4, 5)); // 20

1. Tail Calls and Memoization Tail-call optimisation (TCO) Pfun automatically optimises tail-recursive functions — a recursive call in tail position does not consume stack space. You can recurse to arbitrary depth: function countdown(n) { if n <= 0 then return "Liftoff!" else countdown(n - 1); }

println(countdown(100000)); // Liftoff! (no stack overflow) A call is in tail position when it is the last thing the function does before returning — the result of the call is returned directly, without further computation. Accumulator pattern for loop-like recursion: function sum(lst, acc) { if lst == [] then return acc else sum(tail(lst), acc + head(lst)); }

println(sum([1, 2, 3, 4, 5], 0)); // 15 Memoization Pure functions cache their results. If you call factorial(20, 1) twice, the second call returns the cached value instantly. This is transparent — you never need to manage the cache manually. This makes functions like Fibonacci efficient without any explicit memoization code: function fib(n) { if n <= 1 then return n else fib(n - 1) + fib(n - 2); }

println(fib(30)); // 832040 (fast due to memoization)

1. Lists Lists are immutable and homogeneous — all elements must be the same type. let nums = [1, 2, 3, 4, 5]; let words = ["hello", "world"]; let empty = []; Mixing types in a list is an error: let bad = [1, "two", 3]; // [Type] Error: // Type mismatch in list: expected bigint, got string. Core list operations FunctionDescriptionhead(list)First element. Errors on empty list.tail(list)All elements except the first.cons(x, list)Prepend x to list.map(f, list)Apply f to every element, return new list.filter(f, list)Keep elements where f returns true.reduce(f, init, list)Fold list left with accumulator starting at init.take(n, list)First n elements. Works on infinite lists too.slice(start, count, list)count elements beginning at index start (0-based).nth(list, n)Element at index n, or false if out of bounds. let nums = [1, 2, 3, 4, 5];

println(head(nums)); // 1 println(tail(nums)); // [2, 3, 4, 5] println(cons(0, nums)); // [0, 1, 2, 3, 4, 5]

let evens = filter(fn x => x % 2 == 0, nums); let doubled = map(fn x => x * 2, evens); let total = reduce(fn acc, x => acc + x, 0, doubled);

println(doubled); // [4, 8] println(total); // 12

println(slice(1, 3, nums)); // [2, 3, 4] println(nth(nums, 2)); // 3 println(nth(nums, 99)); // false

1. List Comprehensions [body for var <- source] [body for var <- source where guard] [body for var <- source for var2 <- source2] Comprehensions are a concise way to transform and filter lists. They are pure expressions and can appear anywhere a list is valid. let nums = [1, 2, 3, 4, 5];

// Transform every element let doubled = [x * 2 for x <- nums]; println(doubled); // [2, 4, 6, 8, 10]

// Filter with where let big = [x for x <- nums where x > 3]; println(big); // [4, 5]

// Transform and filter together let big_doubled = [x * 2 for x <- nums where x > 3]; println(big_doubled); // [8, 10]

// Multiple conditions with && let mid = [x for x <- [1..10] where x > 3 && x < 8]; // (written out explicitly since ranges aren't built-in) let mid = [x for x <- [1,2,3,4,5,6,7,8,9,10] where x > 3 && x < 8]; println(mid); // [4, 5, 6, 7] Multiple generators — cartesian product When you use more than one for clause, you get the cartesian product of the sources. The outermost generator varies slowest: let xs = [1, 2, 3]; let ys = [10, 20]; let pairs = [x + y for x <- xs for y <- ys]; println(pairs); // [11, 21, 12, 22, 13, 23] Flattening a matrix Multiple generators can flatten nested lists: let matrix = [[1, 2], [3, 4], [5, 6]]; let flat = [x for row <- matrix for x <- row]; println(flat); // [1, 2, 3, 4, 5, 6] Inside functions Comprehensions are pure and can be used freely inside function bodies: function evens(lst) { return [x for x <- lst where x % 2 == 0]; }

println(evens([1, 2, 3, 4, 5, 6])); // [2, 4, 6]

1. Infinite Lists Pfun supports lazy infinite lists. No values are computed until you materialise some of them with take. Constructors FunctionDescriptioniterate(f, seed)[seed, f(seed), f(f(seed)), ...]repeat(x)[x, x, x, ...] infinitelycycle(list)Repeat a finite list forever let nats = iterate(fn x => x + 1, 1); let powers = iterate(fn x => x * 2, 1); let ones = repeat(1); let lights = cycle(["red", "amber", "green"]);

println(take(5, nats)); // [1, 2, 3, 4, 5] println(take(6, powers)); // [1, 2, 4, 8, 16, 32] println(take(4, ones)); // [1, 1, 1, 1] println(take(7, lights)); // [red, amber, green, red, amber, green, red] Operations on lazy lists map, filter, cons, and tail all work on infinite lists and return new lazy lists — no elements are computed yet: let evens = filter(fn x => x % 2 == 0, nats); println(take(5, evens)); // [2, 4, 6, 8, 10]

let doubled = map(fn x => x * 2, nats); println(take(5, doubled)); // [2, 4, 6, 8, 10] Chain operations freely: // Multiples of 3 from doubled naturals let result = take(5, filter(fn x => x % 3 == 0, map(fn x => x * 2, nats))); println(result); // [6, 12, 18, 24, 30] Materialising with take and slice println(take(5, nats)); // [1, 2, 3, 4, 5] println(slice(10, 5, nats)); // [11, 12, 13, 14, 15] println(nth(nats, 99)); // 100 reduce on infinite lists reduce requires a finite list. Use take first: let sum = reduce(fn acc, x => acc + x, 0, take(10, nats)); println(sum); // 55 Calling reduce directly on an infinite list is an error:

reduce(fn acc, x => acc + x, 0, nats); // [Runtime] Error: // reduce cannot be used on an infinite list. Use take() first. Fibonacci A classic demonstration using iterate with a pair record: type Pair = { a, b } let fibs = map(fn p => p.a, iterate(fn p => Pair { p.b, p.a + p.b }, Pair { 0, 1 }));

println(take(10, fibs)); // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34] Checking if a list is infinite println(isInfinite(nats)); // true println(isInfinite([1, 2, 3])); // false

1. Strings and Characters Chars Char literals use single quotes: 'a', 'Z', '\n', '\t'. let c = 'A'; println(asc©); // 65 (ASCII code) println(chr(97)); // a (char from ASCII code) asc and chr convert between chars and integers: function toUpper© { return chr(asc© - 32); } println(toUpper('h')); // H Strings as lists Strings are sequences of chars. All list operations work on strings: let word = "hello";

println(head(word)); // h (a char) println(tail(word)); // ello (a string) println(cons('H', tail(word))); // Hello

let no_vowels = filter(fn c => c != 'a' && c != 'e' && c != 'i' && c != 'o' && c != 'u', "beautiful"); println(no_vowels); // btfl map over a string returns a string when the function returns chars: function shiftUp© { return chr(asc© + 1); } println(map(fn c => shiftUp©, "abc")); // bcd reduce over a string works character by character: let length = reduce(fn acc, _ => acc + 1, 0, "hello world"); println(length); // 11 String concatenation Use + to concatenate strings. Integers and booleans are automatically converted to strings when concatenated: let name = "Alice"; let age = 30; println(name + " is " + age + " years old"); // Alice is 30 years old Output functions FunctionBehaviourprintln(x)Print x followed by a newlineprint(x)Print x with no newlineprintf("template\n")Interpolated print (see below) All output functions are procedure-only — they cannot be used inside pure function bodies. printf interpolation printf accepts a string with {name} or {record.field} placeholders: let name = "Alice"; let age = 30; printf("Name: {name}, Age: {age}\n"); // Name: Alice, Age: 30

type Point = { x, y } let p = Point { 3, 4 }; printf("Point: {p.x}, {p.y}\n"); // Point: 3, 4 Use { and } for literal brace characters: printf("Use { and } for literal braces\n"); // Use { and } for literal braces printf does not append a newline automatically — include \n explicitly.

1. Record Types type TypeName = { field1, field2, field3 } Records are product types — a named bundle of fields. Declare the type once, then construct instances positionally or by name. type Point = { x, y, z }; type User = { name, age, active }; Construction Three equivalent syntaxes: // Positional (fields assigned in declaration order) let p = Point { 10, 20, 30 };

// Named via braces (any order) let u1 = User { name="Alice", age=30, active=true };

// Named via parens (any order) let u2 = User(age=25, active=false, name="Bob"); Field access println(p.x); // 10 println(u1.name + " is " + u1.age); // Alice is 30 Type consistency Once a record type has been instantiated with a particular field type, that type is fixed for all future instances: type Item = { name, price }

let a = Item { "apple", 100 }; let b = Item { "bread", "free" }; // [Type] Error: // Type mismatch in Item: field 'price' expected bigint, got string.

1. Discriminated Union Types type UnionName = { | VariantA: field1, field2 | VariantB: field1 | VariantC } A discriminated union groups several named variants under one type name. Each variant can have its own set of fields, or no fields at all. type Shape = { | Square: side | Circle: radius | Rectangle: x, y } Construction Variants are constructed the same three ways as plain records: var sq = Square { 10 }; var ci = Circle { radius = 5 }; var re = Rectangle(x=3, y=4); Variants with no fields are bare identifiers: type Direction = { | North | South | East | West }

let d = North; Field access println(sq.side); // 10 println(ci.radius); // 5 println(re.x); // 3 Type consistency per variant Type consistency is enforced per variant, independently: var s1 = Square { 10 }; var s2 = Square { "ten" }; // [Type] Error: // Type mismatch in Square: field 'side' expected bigint, got string. Different variants can use the same field name with different types — they are completely independent schemas: type Dual = { | A: value | B: value } var a = A { 1 }; var b = B { "hello" }; // Fine — A and B are independent

1. Pattern Matching match expr { | VariantName binding -> result | VariantName binding where guard -> result | VariantName _ -> result | _ -> result } match dispatches on the runtime variant of a union value. It is an expression — it produces a value and can appear anywhere a value is valid. let area = match sq { | Square s -> s.side * s.side | Circle c -> c.radius * c.radius | Rectangle r -> r.x * r.y };

println(area); // 100 Bindings The identifier after the variant name is bound to the matched value for the duration of that arm. Use _ to discard it: let kind = match ci { | Square _ -> "square" | Circle _ -> "circle" | Rectangle _ -> "rectangle" };

println(kind); // circle Where guards Add a where clause to filter by a condition. If the guard is false, the arm is skipped and matching continues with the next arm: let label = match ci { | Circle c where c.radius > 10 -> "big circle" | Circle c where c.radius > 2 -> "medium circle" | Circle _ -> "small circle" | Square s -> "square" | Rectangle r -> "rectangle" };

println(label); // medium circle (radius=5, so first guard fails) Wildcard arm | _ -> matches any variant and satisfies exhaustiveness: let fallback = match re { | Square s -> s.side | _ -> 0 };

println(fallback); // 0 (re is a Rectangle) Exhaustiveness Without a wildcard, every variant of the union must be covered. Missing arms are caught at runtime: match sq { | Square s -> s.side | Circle c -> c.radius // Rectangle is missing }; // [Exhaustiveness] Error: // Non-exhaustive match on 'Shape': missing arm(s) for 'Rectangle'. If all guarded arms for a matched variant fail and no wildcard exists: var ci = Circle { 1 }; match ci { | Circle c where c.radius > 10 -> c.radius | Square s -> s.side | Rectangle r -> r.x }; // [Exhaustiveness] Error: // Non-exhaustive match: no arm matched value of type 'Circle'. Match as a nested expression match can appear inside any expression: let doubled_area = (match sq { | Square s -> s.side * s.side | _ -> 0 }) * 2;

println(doubled_area); // 200 Match inside functions match is pure and can be used freely inside function bodies: function describe(shape) { return match shape { | Square s -> "Square with side " + s.side | Circle c -> "Circle with radius " + c.radius | Rectangle r -> "Rectangle " + r.x + "x" + r.y }; }

println(describe(Square { 6 })); // Square with side 6 println(describe(Circle { radius=3 })); // Circle with radius 3

1. The Option Type Option is a built-in union type representing a value that may or may not exist. Its two variants are always available — no type declaration needed. Some { value } // wraps a value None // represents absence let present = Some { 42 }; let absent = None; Use

match to safely unwrap: let result = match present { | Some s -> s.value | None -> 0 };

println(result); // 42 Returning Option from functions Option is the idiomatic way to represent failure in pure functions: function safeDivide(a, b) { return b == 0 ? None : Some { a / b }; }

println(match safeDivide(10, 2) { | Some s -> “Result: ” + s.value | None -> “Division by zero” }); // Result: 5

println(match safeDivide(10, 0) { | Some s -> “Result: ” + s.value | None -> “Division by zero” }); // Division by zero Where guards on Option let x = Some { 100 };

let label = match x { | Some s where s.value > 50 -> “big” | Some _ -> “small” | None -> “nothing” };

println(label); // big Option must be exhaustively matched Because Option is a union type, both arms are required (or a wildcard): let x = Some { 1 }; match x { | Some s -> s.value }; // [Exhaustiveness] Error: // Non-exhaustive match on ‘Option’: missing arm(s) for ‘None’.

1. Dictionaries Dictionaries are mutable key-value stores. Keys must be strings, integers, or booleans. Values can be anything. Dictionaries must always be declared with var. var scores = dict { “Alice” -> 95, “Bob” -> 87 }; var empty = dict {}; var byNum = dict { 1 -> “one”, 2 -> “two” }; Access and mutation println(scores[“Alice”]); // 95

scores[“Bob”] = 90; // update scores[“Carol”] = 88; // add new key remove(scores, “Carol”); // delete key Accessing a key that does not exist is an error: eval scores[“missing”]; // [Key] Error: // Key not found in dict: “missing” Dict builtins FunctionDescriptionhas(d, key)true if key existsremove(d, key)Delete a key (no-op if missing)keys(d)List of all keysvalues(d)List of all values var d = dict { “x” -> 10, “y” -> 20 };

println(has(d, “x”)); // true println(has(d, “z”)); // false println(keys(d)); // [x, y] println(values(d)); // [10, 20] Dicts in procedures Dicts are imperative — they can only be mutated inside procedures or at the top level. Attempting to mutate a dict inside a pure function is an error: function bad(d) { d[“x”] = 1; return d; } // [Purity] Error: // Functions cannot mutate dicts.

1. Search and Access find find(list, item) // returns first index, or -1 Uses deep value equality — records and nested lists are compared by content, not by reference. let nums = [10, 20, 30, 40, 50]; println(find(nums, 30)); // 2 println(find(nums, 99)); // -1

// First match only println(find([1, 2, 1, 3], 1)); // 0

// Char search in a string println(find(“hello”, ‘l’)); // 2

// Record search by value type Point = { x, y } let pts = [Point { 1, 2 }, Point { 3, 4 }, Point { 5, 6 }]; println(find(pts, Point { 3, 4 })); // 1 findSlice findSlice(list, sublist) // returns start index, or -1 println(findSlice([1, 2, 3, 4, 5], [2, 3, 4])); // 1 println(findSlice([1, 2, 3], [4, 5])); // -1 println(findSlice([1, 2, 3], [])); // 0 (empty always matches at 0)

// Substring search println(findSlice(“hello world”, “world”)); // 6 println(findSlice(“hello world”, “xyz”)); // -1 Neither find nor findSlice works on infinite lists: find(iterate(fn x => x + 1, 1), 5); // [Runtime] Error: // find/findSlice cannot search an infinite list. Use take() first.

1. Input and Output All I/O is procedure-only. Attempting to use any I/O function inside a pure function produces a [Purity] error. To use I/O in a script, import the io module: import * from “io”; Output println(x) // print x with a newline print(x) // print x without a newline printf(“...”) // interpolated print (see Section 14) print and println accept any value and convert it to a string automatically. Input from stdin readChar() // reads one character — returns Some { char } or None at EOF readln() // reads one line (newline stripped) — returns Some { string } or None at EOF Both return an Option, which you unwrap with match: proc prompt() { print(“Enter your name: ”); var input = readln(); let name = match input { | Some s -> s.value | None -> “stranger” }; printf(“Hello, {name}!\n”); }

prompt(); Echo loop proc echoLoop() { var line = readln(); match line { | Some s -> { println(“Got: ” + s.value); echoLoop(); } | None -> println(“EOF”); }; } Reading from a pipe echo “friend” | npm start myscript.pf

1. File I/O Import the file module to use file operations: import * from “file”; All file functions are procedure-only. Whole-file convenience functions readFile(path) // returns Some { string } or None writeFile(path, content) // returns Some { bytes_written } or None proc copyFile(src, dst) { let content = readFile(src); match content { | Some s -> writeFile(dst, s.value) | None -> println(“Could not read: ” + src) }; } Handle-based I/O For reading or writing incrementally, use file handles: fileOpen(path, mode) // mode is “r” or “w” — returns Some { handle } or None fileClose(handle) // close the handle The handle is a FileHandle union value — either a ReadHandle or WriteHandle. The read/write functions check this at runtime and will error if you pass the wrong kind. Reading: readChar(handle) // Some { char } or None at EOF readLine(handle) // Some { string } or None at EOF (newline stripped) Writing: writeChar(handle, char) // returns Some { bytes } or None writeLine(handle, string) // writes string + newline, returns Some { bytes } or None Example — read a file line by line: proc printLines(path) { var h = fileOpen(path, “r”); match h { | Some s -> { var handle = s.value; var line = readLine(handle); match line { | Some l -> { println(l.value); readLine(handle); } | None -> 0 }; fileClose(handle); } | None -> println(“Could not open: ” + path) }; }

2. Modules and Imports Pfun supports multi-file projects with named and namespace imports. Exporting from a module Any top-level declaration can be exported by prefixing it with export: // mathutils.pf

export function add(x, y) { return x + y; } export function multiply(x, y) { return x * y; } export let tau = 6; export proc printResult(label, value) { println(label + “: ” + value); } Only explicitly exported names are importable. A module’s private let or function bindings are not visible to importers. Named imports import { add, multiply } from “./mathutils”;

println(add(3, 4)); // 7 println(multiply(3, 4)); // 12 Import with an alias: import { add as plus } from “./mathutils”;

println(plus(3, 4)); // 7 Namespace imports Bring all exports in under a single name: import * as Math from “./mathutils”;

println(Math.add(10, 5)); // 15 println(Math.tau); // 6 Math.printResult(“x”, 42); // x: 42 Star imports Import everything directly into the current

scope (used for built-in modules): import * from “io”; import * from “file”; Built-in modules ModuleContents"io"print, println, printf, readChar, readln"file"readFile, writeFile, fileOpen, fileClose, readChar, readLine, writeChar, writeLine Module resolution

Paths starting with ./ or ../ are resolved relative to the importing file. Bare names (no /) are resolved from a lib/ subdirectory alongside the script. Each module is executed once and cached — importing the same module from multiple files does not re-execute it. Circular imports are detected and produce an [Import] error.

Common import errors import { secret } from “./mod”; // [Import] Error: // Module ‘./mod.pf’ does not export ‘secret’.

import { a } from “./a”; // where a.pf imports b.pf which imports a.pf // [Import] Error: // Circular import detected: /path/to/a.pf

1. Error Reference Pfun errors are prefixed with their kind, followed by the line and column, the source line, a caret, and (where applicable) the values of identifiers referenced on that line. [Kind] Error on line N/chM: Error message. source line text ^

identifier = value Error kinds KindMeaning[Lexical]Unexpected character, unterminated literal, bad escape sequence[Syntax]Unexpected token, missing keyword or bracket[Name]Undefined variable, unknown type, immutable reassignment, missing property[Type]Type mismatch in record field, list, or builtin argument[Key]Missing dict key, missing record field, list index out of bounds[DivideByZero]Division or modulo by zero[Purity]Side effect inside a pure function: var, print, proc call, dict mutation[Exhaustiveness]Non-exhaustive match — missing variant arm or all guards failed[Arity]Wrong number of fields in record constructor[Import]Module not found, circular import, missing named export[File]File not found or permission error[IO]stdin/stdout error[Runtime]Everything else: head on empty list, reduce on infinite list, etc. Selected examples [Lexical] — unexpected character let x = @; // [Lexical] Error on line 1/ch9: // Unexpected character ‘@’ // let x = @; // ^ [Name] — undefined variable let result = x + y; // [Name] Error on line 1/ch14: // Undefined variable ‘y’. // let result = x + y; // ^ // // x = 10 // result = [Type] — list type mismatch let l = [1, “two”, 3]; // [Type] Error: // Type mismatch in list: expected bigint, got string. [Key] — missing dict key var d = dict { “a” -> 1 }; eval d[“b”]; // [Key] Error on line 2/ch6: // Key not found in dict: “b” // eval d[“b”]; // ^ // // d = dict { … } [DivideByZero] let x = 10; let y = 0; eval x / y; // [DivideByZero] Error on line 3/ch6: // Divide by zero. // eval x / y; // ^ // // x = 10 // y = 0 [Purity] — print in a function function bad(x) { println(x); } // [Purity] Error on line 2/ch5: // Functions cannot use ‘println’: side effects are not allowed // in pure functions. // println(x); // ^ // // x = [Exhaustiveness] — missing match arm type Shape = { | Square: side | Circle: radius | Rectangle: x, y } var sq = Square { 4 }; match sq { | Square s -> s.side | Circle c -> c.radius }; // [Exhaustiveness] Error: // Non-exhaustive match on ‘Shape’: missing arm(s) for ‘Rectangle’. [Arity] — wrong field count type Point = { x, y } var p = Point { 1 }; // [Arity] Error: // ‘Point’ expects 2 field(s), got 1. [Import] — missing export import { secret } from “./mod”; // [Import] Error: // Module ‘./mod’ does not export ‘secret’.

Pfun 0.1 — language and manual subject to change.